

Competitive Programming Notebook

py.Sandu

Contents

1 Geometry	2	
1.1 Cross Product	2	
1.2 Convex Hull	2	
1.3 Isleft	2	
1.4 Winding Number	2	
1.5 Point	2	
2 DS	2	
2.1 Union Find	2	
2.2 Trie	3	
2.3 Fenwick Tree	3	
2.4 Monotonic Stack	3	
2.5 Seg Tree	4	
3 General	4	
3.1 Mex	4	
3.2 Josephus	5	
4 String	5	
4.1 Kmp	5	
4.2 Duval	5	
5 Math	5	
5.1 Siege Of Eratosthenes	5	
5.2 Matrix Fibonacci	6	
5.3 Matrix Multiplication	6	
5.4 Solve Linear System	6	
5.5 Modular Exponentiation	6	
5.6 Spf	7	
5.7 Gcd	7	
5.8 Extended Euclidean Algorithm	7	
6 Graph	7	
6.1 Bellman-ford	7	
6.2 Floyd-warshall	7	
6.3 Ford-fulkerson	7	
6.4 Mst Prim	8	
6.5 Dfs	8	
6.6 Bfs	8	
6.7 Reverse Graph	9	
6.8 Eulerean Path	9	
6.9 Djikstra	9	
7 DP	9	
7.1 Binary Lifting	9	
7.2 Knapsack	10	
7.3 Coins Change	10	
7.4 Longest Increasing Subsequence	10	
		7.5 Kadane Algorithm 10

1 Geometry

1.1 Cross Product

```
1 double cross_product(Point O, Point A, Point B)
2 {
3     return (A.x - O.x) * (B.y - O.y)
4           - (A.y - O.y) * (B.x - O.x);
5 }
```

1.2 Convex Hull

```
1 // Returns a list of points on the convex hull
2 // in counter-clockwise order
3 vector<Point> convex_hull(vector<Point> A)
4 {
5     int n = A.size(), k = 0;
6
7     if (n <= 3)
8         return A;
9
10    vector<Point> ans(2 * n);
11
12    // Sort points lexicographically
13    sort(A.begin(), A.end());
14
15    // Build lower hull
16    for (int i = 0; i < n; ++i) {
17
18        // If the point at K-1 position is not a part
19        // of hull as vector from ans[k-2] to ans[k
20        -1]
21        // and ans[k-2] to A[i] has a clockwise turn
22        while (
23            k >= 2
24            && cross_product(ans[k - 2], ans[k - 1],
25                A[i])
26                <= 0)
27            k--;
28            ans[k++] = A[i];
29    }
30
31    // Build upper hull
32    for (size_t i = n - 1, t = k + 1; i > 0; --i) {
33
34        // If the point at K-1 position is not a part
35        // of hull as vector from ans[k-2] to ans[k
36        -1]
37        // and ans[k-2] to A[i] has a clockwise turn
38        while (k >= t
39            && cross_product(ans[k - 2], ans[k -
40                1],
41                A[i - 1])
42                <= 0)
43            k--;
44            ans[k++] = A[i - 1];
45    }
46
47    // Resize the array to desired size
48    ans.resize(k - 1);
49
50    return ans;
51 }
```

1.3 Isleft

```
1 // Check if a point is to the left of a segment
2 // defined by two points
3 bool isLeft(const Point& p, const Point& v1, const
4     Point& v2) {
5     return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
6     y) * (p.x - v1.x) > 0;
7 }
```

```
4 }
```

1.4 Winding Number

```
1 // Check if a point is inside a polygon using the
2 // winding number algorithm
3 bool isInsidePolygon(const Point& p, const vector<
4     Point>& polygon) {
5     int windingNumber = 0;
6     int n = polygon.size();
7
8     for (int i = 0; i < n; i++) {
9         const Point& v1 = polygon[i];
10        const Point& v2 = polygon[(i + 1) % n];
11
12        if (v1.y <= p.y) {
13            if (v2.y > p.y && isLeft(p, v1, v2)) {
14                ++windingNumber;
15            }
16        } else {
17            if (v2.y <= p.y && !isLeft(p, v1, v2)) {
18                --windingNumber;
19            }
20        }
21    }
22
23    return windingNumber != 0;
24 }
```

1.5 Point

```
1 struct Point {
2     double x, y;
3 };
```

2 DS

2.1 Union Find

```
1 // Time complexity:  $\mathcal{O}(1)$  for  $n < 10^600$ 
2 // Memory complexity:  $\mathcal{O}(n)$ 
3
4 class UnionFind{
5     vector<int> parent;
6     vector<int> setSize;
7 public:
8     UnionFind(int size){
9         parent.resize(size);
10        setSize.assign(size, 1);
11        iota(parent.begin(), parent.end(), 0); // If
12        needed to index by 1, size = size+1
13    }
14
15    int find(int i){
16        return parent[i] == i ? i : parent[i] = find(
17        parent[i]);
18    }
19
20    bool join(int I, int J){
21        int i = find(I), j = find(J);
22        if(i == j) return false;
23
24        if(setSize[i] < setSize[j]) swap(i, j);
25
26        parent[j] = i;
27
28        setSize[i] += setSize[j];
29        return true;
30    }
31 }
```

```

30     int size(int i){ return setSize[find(i)]; }
31 };

```

2.2 Trie

```

1 // class for a node of the trie
2 class TrieNode {
3 public:
4     bool endofWord;
5     TrieNode* children[26];
6
7     // Constructore to initialize a trie node
8     TrieNode()
9     {
10         endofWord = false;
11         for (int i = 0; i < 26; i++) {
12             children[i] = nullptr;
13         }
14     }
15 };
16
17 // class for the Trie implementation
18 class Trie {
19 private:
20     TrieNode* root;
21
22 public:
23     Trie() { root = new TrieNode(); }
24
25     // Function to insert a word into the trie
26     void insert(string word)
27     {
28         TrieNode* node = root;
29         for (char c : word) {
30             int index = c - 'a';
31             if (!node->children[index]) {
32                 node->children[index] = new TrieNode
33             }
34             node = node->children[index];
35         }
36         node->endofWord = true;
37     }
38
39     // Function to search a word in the trie
40     bool search(string word)
41     {
42         TrieNode* node = root;
43         for (char c : word) {
44             int index = c - 'a';
45             if (!node->children[index]) {
46                 return false;
47             }
48             node = node->children[index];
49         }
50         return node->endofWord;
51     }
52
53     // Function to check if there is any word in the
54     // trie that starts with the given prefix
55     bool startsWith(string prefix)
56     {
57         TrieNode* node = root;
58         for (char c : prefix) {
59             int index = c - 'a';
60             if (!node->children[index]) {
61                 return false;
62             }
63             node = node->children[index];
64         }
65         return true;
66     }

```

```

67
68     // Function to delete a word from the trie
69     void deleteWord(string word)
70     {
71         TrieNode* node = root;
72         for (char c : word) {
73             int index = c - 'a';
74             if (!node->children[index]) {
75                 return;
76             }
77             node = node->children[index];
78         }
79         if (node->endofWord == true) {
80             node->endofWord = false;
81         }
82     }
83
84     // Function to print the trie
85     void print(TrieNode* node, string prefix) const
86     {
87         if (node->endofWord) {
88             cout << prefix << endl;
89         }
90         for (int i = 0; i < 26; i++) {
91             if (node->children[i]) {
92                 print(node->children[i],
93                     prefix + char('a' + i));
94             }
95         }
96     }
97
98     // Function to start printing from the root
99     void print() const { print(root, ""); }
100 };

```

2.3 Fenwick Tree

```

1 // Fenwick Tree to efficiently calculate and update
2 // prefix sums of an array of values.
3 // Time complexity: O(log N) for queries and updates
4
5 int sum(int i) {
6     int sum = 0;
7     while(i > 0) {
8         sum += T[i];
9         i -= i & -i; // flip the last bit
10    }
11    return sum;
12 }
13
14 void add(int i, int k) {
15     while (i < T.size()) {
16         T[i] += k;
17         i += i & -i; // add last set bit
18    }
19 }
20
21 vi make(vi arr) {
22     vi tree = arr;
23
24     for(int i = 0 ; i < tree.size() ; i++) {
25         int p = i + (i & -i) // index to parent
26         if (p < t.size()) {
27             tree[p] = tree[p] + tree[i];
28         }
29     }
30     return tree;
31 }

```

2.4 Monotonic Stack

```

1 // Increasing Monotonic Stack

```

```

2 // Time Complexity: O(n)
3 // Space Complexity: O(n)
4
5 vector<int> nextGreaterElements(vector<int>& arr) {
6     int n = arr.size();
7     vector<int> res(n, -1);
8     stack<int> st;
9
10    for (int i = 0; i < 2 * n; i++) {
11        int num = arr[i % n];
12        while (!st.empty() && arr[st.top()] < num) {
13            res[st.top()] = num;
14            st.pop();
15        }
16        if (i < n) {
17            st.push(i);
18        }
19    }
20
21    return res;
22 }
23
24 // Decreasing Monotonic Stack
25 // Time Complexity: O(n)
26 // Space Complexity: O(n)
27
28 vector<int> nextSmallerElements(vector<int>& arr) {
29     int n = arr.size();
30     vector<int> res(n, -1);
31     stack<int> st;
32
33    for (int i = 0; i < 2 * n; ++i) {
34        int num = arr[i % n];
35        while (!st.empty() && arr[st.top()] > num) {
36            res[st.top()] = num;
37            st.pop();
38        }
39        if (i < n) {
40            st.push(i);
41        }
42    }
43
44    return res;
45 }

```

2.5 Seg Tree

```

1 // Segment Tree implementation
2 // Time complexity: O(log n) for updates and queries
3
4 template<typename T>
5 class SegmentTree {
6 private:
7     int n;
8     vector<T> tree;
9     T combine(T a, T b) {
10         // Combine function for the segment tree (e.g
11         // sum, min, max)
12         return a + b; // Example: sum
13     }
14 void build(vector<T>& arr, int v, int tl, int tr)
15     {
16         if (tl == tr) {
17             tree[v] = arr[tl];
18         } else {
19             int tm = (tl + tr) / 2;
20             build(arr, v * 2, tl, tm);
21             build(arr, v * 2 + 1, tm + 1, tr);
22             tree[v] = combine(tree[v * 2], tree[v * 2

```

```

23         void update(int v, int tl, int tr, int pos, T
24         new_val) {
25             if (tl == tr) {
26                 tree[v] = new_val;
27             } else {
28                 int tm = (tl + tr) / 2;
29                 if (pos <= tm) {
30                     update(v * 2, tl, tm, pos, new_val);
31                 } else {
32                     update(v * 2 + 1, tm + 1, tr, pos,
33                     new_val);
34                 }
35                 tree[v] = combine(tree[v * 2], tree[v * 2
36                 + 1]);
37             }
38         }
39         T query(int v, int tl, int tr, int l, int r) {
40             if (l > r) {
41                 return T(); // Return identity element (e
42                 .g., 0 for sum)
43             }
44             if (l == tl && r == tr) {
45                 return tree[v];
46             }
47             int tm = (tl + tr) / 2;
48             return combine(query(v * 2, tl, tm, l, min(r,
49             tm)),
50             query(v * 2 + 1, tm + 1, tr,
51             max(l, tm + 1), r));
52         }
53     public:
54     SegmentTree(vector<T>& arr) {
55         n = arr.size();
56         tree.resize(4 * n);
57         build(arr, 1, 0, n - 1);
58     }
59     void update(int pos, T new_val) {
60         update(1, 0, n - 1, pos, new_val);
61     }
62     T query(int l, int r) {
63         return query(1, 0, n - 1, l, r);
64     }
65 };

```

3 General

3.1 Mex

```

1 // Minimal excluded number of a sequence
2
3 // O(N log N) approach
4 int mex(vector<int> const& A) {
5     set<int> b(A.begin(), A.end());
6
7     int result = 0;
8     while (b.count(result))
9         ++result;
10    return result;
11 }
12
13 // O(N) approach
14 int mex(vector<int> const& A) {
15     static bool used[MAX_N+1] = { 0 };
16
17     // mark the given numbers
18     for (int x : A) {
19         if (x <= MAX_N)
20             used[x] = true;
21     }
22
23     // find the mex
24     int result = 0;

```

```

25 while (used[result])
26     ++result;
27
28 // clear the array again
29 for (int x : A) {
30     if (x <= MAX_N)
31         used[x] = false;
32 }
33
34 return result;
35 }

```

3.2 Josephus

```

1 // Josephus problem
2 // Time complexity: O(N)
3 // Memory complexity: O(1)
4
5 // Recursive (1 indexing)
6 int memo[MAX_N];
7
8 int josephus(int n, int k) {
9     if (n == 1) return 1;
10    if (memo[n]) return memo[n];
11    return memo[n] = (josephus(n-1, k) + k - 1) % n +
12    1;
13 }
14
15 // Non-Recursive
16 int josephus(int n, int k) {
17     int res = 0;
18     for (int i = 1; i <= n; ++i)
19         res = (res + k) % i;
20     return res + 1;
21 }

```

4 String

4.1 Kmp

```

1 // Find all occurrences of a pattern in a given text
2 // using the Knuth-Morris-Pratt (KMP) algorithm
3 // Time complexity: O(n + m) where n is the length of
4 // the text and m is the length of the pattern
5
6 vector<int> LPS(const string& pattern) {
7     int m = pattern.size();
8     vector<int> lps(m, 0);
9     int len = 0; // length of the previous longest
10    prefix suffix
11    int i = 1;
12
13    while (i < m) {
14        if (pattern[i] == pattern[len]) {
15            len++;
16            lps[i] = len;
17            i++;
18        } else {
19            if (len != 0) {
20                len = lps[len - 1];
21            } else {
22                lps[i] = 0;
23                i++;
24            }
25        }
26    }
27    return lps;
28 }
29
30 vector<int> KMP(const string& text, const string&
31 pattern) {

```

```

28 vector<int> lps = LPS(pattern);
29 vector<int> occurrences;
30 int n = text.size();
31 int m = pattern.size();
32 int i = 0; // index for text
33 int j = 0; // index for pattern
34
35 while (i < n) {
36     if (text[i] == pattern[j]) {
37         i++;
38         j++;
39     }
40     if (j == m) {
41         occurrences.push_back(i - j); // Pattern
42         found at index (i - j)
43         j = lps[j - 1]; // Continue to search for
44         the next match
45     } else if (i < n && text[i] != pattern[j]) {
46         if (j != 0) {
47             j = lps[j - 1];
48         } else {
49             i++;
50         }
51     }
52 }
53 return occurrences;
54 }

```

4.2 Duval

```

1 // Duval algorithm for factorizing strings in
2 // lexicographical order
3 // Time complexity: O(n)
4 // Memory complexity: O(n)
5
6 vector<string> duval(string const& s) {
7     int n = s.size();
8     int i = 0;
9     vector<string> factorization;
10    while (i < n) {
11        int j = i + 1, k = i;
12        while (j < n && s[k] <= s[j]) {
13            if (s[k] < s[j])
14                k = i;
15            else
16                k++;
17            j++;
18        }
19        while (i <= k) {
20            factorization.push_back(s.substr(i, j - k
21            ));
22            i += j - k;
23        }
24    }
25    return factorization;
26 }

```

5 Math

5.1 Sieve Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
2 // the Sieve of Eratosthenes algorithm
3 // Time complexity: O(n log log n) where n is the
4 // limit
5
6 vector<int> findPrimes(int limit) {
7     vector<bool> isPrime(limit + 1, true);
8     isPrime[0] = isPrime[1] = false;
9
10    for (int i = 2; i * i <= limit; i++) {

```

```

9         if (isPrime[i]) {
10             for (int j = i * i; j <= limit; j += i) {
11                 isPrime[j] = false;
12             }
13         }
14     }
15
16     vector<int> primes;
17     for (int i = 2; i <= limit; i++) {
18         if (isPrime[i]) {
19             primes.push_back(i);
20         }
21     }
22     return primes;
23 }

```

5.2 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
  // exponentiation.
2 // Time Complexity: O(log N)
3 // Space Complexity: O(1)
4
5 void matrixExponentiation(vector<vector<long long>>&
  matrix, int power) {
6     int n = matrix.size();
7     vector<vector<long long>> result(n, vector<long
  long>(n, 0));
8
9     // Initialize result as the identity matrix
10    for (int i = 0; i < n; i++) {
11        result[i][i] = 1;
12    }
13
14    while (power) {
15        if (power % 2 == 1) {
16            matrixMultiplication(result, matrix,
17            result);
18        }
19        matrixMultiplication(matrix, matrix, matrix);
20        power /= 2;
21    }
22
23    matrix = result;
24 }

```

5.3 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
  // result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>> matrixMultiplication(const vector
  <vector<int>>& A, const vector<vector<int>>& B) {
6     int n = A.size();
7     vector<vector<int>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10        for (int j = 0; j < n; j++) {
11            for (int k = 0; k < n; k++) {
12                C[i][j] += A[i][k] * B[k][j];
13            }
14        }
15    }
16    return C;
17 }

```

5.4 Solve Linear System

```

1 // Solve a system of linear equations using LU
  // decomposition
2 // Time complexity: O(n^3) for decomposition, O(n^2)
  // for forward and backward substitution
3
4 void luDecomposition(const vector<vector<double>>& A,
  vector<vector<double>>& L, vector<vector<double>
  >>& U) {
5     int n = A.size();
6     for (int i = 0; i < n; i++) {
7         for (int j = 0; j < n; j++) {
8             if (i <= j) {
9                 U[i][j] = A[i][j];
10                for (int k = 0; k < i; k++) {
11                    U[i][j] -= L[i][k] * U[k][j];
12                }
13            } else {
14                L[i][j] = A[i][j];
15                for (int k = 0; k < j; k++) {
16                    L[i][j] -= L[i][k] * U[k][j];
17                }
18                L[i][j] /= U[j][j];
19            }
20        }
21    }
22 }
23
24 void forwardSubstitution(const vector<vector<double>
  >>& L, const vector<double>& b, vector<double>& y
  ) {
25     int n = L.size();
26     for (int i = 0; i < n; i++) {
27         y[i] = b[i];
28         for (int j = 0; j < i; j++) {
29             y[i] -= L[i][j] * y[j];
30         }
31     }
32 }
33
34 void backwardSubstitution(const vector<vector<double>
  >>& U, const vector<double>& y, vector<double>& x
  ) {
35     int n = U.size();
36     for (int i = n - 1; i >= 0; i--) {
37         x[i] = y[i];
38         for (int j = i + 1; j < n; j++) {
39             x[i] -= U[i][j] * x[j];
40         }
41         x[i] /= U[i][i];
42     }
43 }
44 }

```

5.5 Modular Exponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int powMod(int x, int n, int MOD){
6     int res = 1;
7
8     while(n >= 1){
9         if(n & 1){
10            res = (res * x) % MOD;
11            n--;
12        }
13        else{
14            x = (x * x) % MOD;
15            n /= 2;
16        }
17    }
18    return res;

```

19 }

5.6 Spf

```

1 // SPF sieve implementation for finding the smallest
  prime factor of each number up to a given limit.
2 // Time complexity:  $O(n \log \log n)$  for the sieve
  construction,  $O(\log n)$  for factorization of each
  number.
3
4 #define MAXN 1000000 // Maximum limit for the sieve
5 std::vector<int> spf(MAXN + 1);
6
7 void sieve() {
8     for (int i = 1; i <= MAXN; i++) {
9         spf[i] = i;
10    }
11    for (int i = 2; i * i <= MAXN; i++) {
12        if (spf[i] == i) {
13            for (int j = i * i; j <= MAXN; j += i) {
14                if (spf[j] == j) {
15                    spf[j] = i;
16                }
17            }
18        }
19    }
20 }
```

5.7 Gcd

```

1 // Calculates the Greatest Common Divisor (GCD) of
  two integers using Euclidean algorithm
2 // Time Complexity:  $O(\log(\min(a, b)))$ 
3 // Space Complexity:  $O(\log(\min(a, b)))$ 
4
5 int gcd(int a, int b){
6     if (a == 0) return b;
7     return gcd(b % a, a);
8 }
```

5.8 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that  $a*x + b*y =$ 
  gcd(a, b)
3
4 int extendedEuclidean(int a, int b, int& x, int& y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10    int x1, y1;
11    int gcd = extendedEuclidean(b % a, a, x1, y1);
12    x = y1 - (b / a) * x1;
13    y = x1;
14    return gcd;
15 }
```

6 Graph

6.1 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
  other vertices in a graph with negative weight
  edges and detect negative weight cycles.
2 // Time Complexity:  $O(V * E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 void bellmanFord(Graph& graph, int src) {
```

```

6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);
8     dist[src] = 0;
9
10    for (int i = 1; i < V; i++) {
11        for (int u = 0; u < V; u++) {
12            for (int v = 0; v < V; v++) {
13                if (graph[u][v] != 0 && dist[u] !=
14                    INT_MAX && dist[u] + graph[u][v] < dist[v]) {
15                    dist[v] = dist[u] + graph[u][v];
16                }
17            }
18        }
19    }
20
21    // Check for negative weight cycles
22    for (int u = 0; u < V; u++) {
23        for (int v = 0; v < V; v++) {
24            if (graph[u][v] != 0 && dist[u] !=
25                INT_MAX && dist[u] + graph[u][v] < dist[v]) {
26                cout << "Graph contains negative
27                    weight cycle" << endl;
28                return;
29            }
30        }
31    }
32 }
```

6.2 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
  positive or negative edge weights (but with no
  negative cycles).
2 // Time Complexity:  $O(V^3)$ 
3 // Space Complexity:  $O(V^2)$ 
4
5 void floydWarshall(vector<vector<int>>& dist) {
6     int V = dist.size();
7     int INF = 1e8;
8
9     for (int k = 0; k < V; k++) {
10        for (int i = 0; i < V; i++) {
11            for (int j = 0; j < V; j++) {
12                if (dist[i][k] != INF && dist[k][j] !=
13                    INF) {
14                    dist[i][j] = min(dist[i][j],
15                                    dist[i][k] +
16                                    dist[k][j]);
17                }
18            }
19        }
20    }
```

6.3 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity:  $O(|V| * E^2)$ 
3 // Space complexity:  $O(V)$ 
4 bool bfs(int residual[V][V], int start, int target,
  vi parent){
5     vector<bool> visited(V, false);
6
7     queue<int> q;
8     q.push(start);
9     visited[start] = true;
10    parent[start] = -1;
11
12    while(!q.empty()){
13        int u = q.front();
14        q.pop();
15    }
```

```

16     for(int v = 0 ; v <= V ; v++){
17         if(!visited[v] and residual[u][v] > 0){
18             if(v == target){
19                 parent[v] = u;
20                 return true;
21             }
22             q.push(v);
23             parent[v] = u;
24             visited[v] = true;
25         }
26     }
27 }
28 return false;
29 }
30
31 int fordFulkerson(int g[V][V], int start, int target)
32 {
33     int u, v;
34     int residual[V][V];
35
36     for(u = 0 ; u < V ; u++)
37         for(v = 0 ; v < V ; v++)
38             residual[u][v] = g[u][v];
39
40     vi parent(V);
41     int maxFlow = 0;
42
43     while(bfs(residual, start, target, parent)){
44         int pathFlow = INT_MAX;
45         for(v = target ; v != start ; v = parent[v]){
46             u = parent[v];
47             pathFlow = min(pathFlow, residual[u][v]);
48         }
49
50         for(v = target ; v != start ; v = parent[v]){
51             u = parent[v];
52             residual[u][v] -= pathFlow;
53             residual[v][u] += pathFlow;
54         }
55         maxFlow += pathFlow;
56     }
57     return maxFlow;
58 }

```

6.4 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
2 // Tree
3 // Time complexity:  $O(E \log V)$  where E is the number
4 // of edges and V is the number of vertices
5
6 struct Edge {
7     int u, v, weight;
8     bool operator<(const Edge& other) const {
9         return weight > other.weight; // Min-heap
10     }
11 };
12
13 vector<Edge> primMST(const vector<vector<int>>& graph)
14 {
15     int n = graph.size();
16     vector<bool> visited(n, false);
17     vector<Edge> mst;
18     priority_queue<Edge> pq;
19
20     // Start with the first vertex
21     visited[0] = true;
22     for (int i = 0; i < n; ++i) {
23         if (graph[0][i] != 0) {
24             pq.push({0, i, graph[0][i]});
25         }
26     }
27 }

```

```

25 while (!pq.empty()) {
26     Edge current = pq.top();
27     pq.pop();
28
29     if (visited[current.v]) continue;
30
31     visited[current.v] = true;
32     mst.push_back(current);
33
34     for (int i = 0; i < n; ++i) {
35         if (!visited[i] && graph[current.v][i] !=
36             0) {
37             pq.push({current.v, i, graph[current.
38                 v][i]});
39         }
40     }
41 }
42 return mst;
43 }

```

6.5 Dfs

```

1 // Depth First Search (DFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
7     & visited) {
8     visited[vertex] = true;
9
10    for (int neighbor : graph[vertex]) {
11        if (!visited[neighbor]) {
12            dfs(graph, neighbor, visited);
13        }
14    }
15 }
16
17 // Iterative DFS
18 void dfsIterative(const Graph& graph, int start) {
19     int V = graph.size();
20     vector<bool> visited(V, false);
21     stack<int> s;
22     s.push(start);
23
24     while (!s.empty()) {
25         int vertex = s.top();
26         s.pop();
27
28         if (!visited[vertex]) {
29             visited[vertex] = true;
30
31             for (int neighbor : graph[vertex]) {
32                 if (!visited[neighbor]) {
33                     s.push(neighbor);
34                 }
35             }
36         }
37     }
38 }

```

6.6 Bfs

```

1 // Breadth First Search (BFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4 void bfs(const Graph& graph, int start) {
5     int V = graph.size();
6     vector<bool> visited(V, false);
7     queue<int> q;
8
9     visited[start] = true;

```



```

10 q.push(start);
11
12 while (!q.empty()) {
13     int vertex = q.front();
14     q.pop();
15     cout << vertex << " ";
16
17     for (int neighbor : graph[vertex]) {
18         if (!visited[neighbor]) {
19             visited[neighbor] = true;
20             q.push(neighbor);
21         }
22     }
23 }
24 }

```

6.7 Reverse Graph

```

1 // Reverse the edges of a directed graph
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V + E)
4
5 Graph reverseGraph(const Graph& graph) {
6     int V = graph.size();
7     Graph reversedGraph(V);
8
9     for (int u = 0; u < V; u++) {
10         for (int v : graph[u]) {
11             reversedGraph[v].push_back(u);
12         }
13     }
14
15     return reversedGraph;
16 }

```

6.8 Eulerian Path

```

1 // Find the Eulerian path in a graph using Hierholzer
  's algorithm
2 // Time complexity: O(E) where E is the number of
  edges in the graph
3
4 void findEulerianPath(const vector<vector<int>>&
  graph, int start, vector<int>& path) {
5     stack<int> s;
6     s.push(start);
7
8     while (!s.empty()) {
9         int v = s.top();
10        if (graph[v].empty()) {
11            path.push_back(v);
12            s.pop();
13        } else {
14            int u = graph[v].back();
15            graph[v].pop_back(); // Remove the edge
  from v to u
16            s.push(u);
17        }
18    }
19 }

```

6.9 Dijkstra

```

1 // Find shortest path from a source vertex to all
  other vertices in a graph with non-negative edge
  weights
2 // Time Complexity: O((V + E) log V)
3 // Space Complexity: O(V)
4
5 void dijkstra(const Graph& graph, int src) {
6     int V = graph.size();
7     vector<int> dist(V, INT_MAX);

```

```

8     dist[src] = 0;
9
10    priority_queue<pair<int, int>, vector<pair<int,
  int>>, greater<pair<int, int>>> pq; // Min-heap
  priority queue
11    pq.push({0, src});
12
13    while (!pq.empty()) {
14        int u = pq.top().second;
15        pq.pop();
16
17        for (const auto& adj : graph[u]) {
18            int v = adj.first;
19            int weight = adj.second;
20
21            if (dist[u] != INT_MAX && dist[u] +
  weight < dist[v]) {
22                dist[v] = dist[u] + weight;
23                pq.push({dist[v], v});
24            }
25        }
26    }
27 }

```

7 DP

7.1 Binary Lifting

```

1 // Binary lifting to find lower common ancestor in a
  binary tree
2 // Time complexity: O(log N) for queries and O(N log
  N) for pre-processing
3 // Memory complexity: O(N)
4 struct BinaryLifting{
5     int n, l, timer; // n = size, l = max jump
6     vector<int> tin, tout;
7     vector<vector<int>> up;
8
9     //Constructor
10    BinaryLifting(int n, int root, const vector<vector
  <int>>& adj) : n(n), timer(0) {
11        l = n <= 1 ? 0 : __lg(n) + 1;
12        tin.resize(n);
13        tout.resize(n);
14        up.assign(n, vector<int>(l+1));
15
16        dfs(root, root, adj);
17    }
18
19    void dfs(int v, int p, const vector<vector<int>>&
  adj){
20        tin[v] = ++timer;
21        up[v][0] = p;
22
23        for(int i = 1 ; i <= l ; i++) up[v][i] = up[
  up[v][i-1]][i-1];
24
25        for(auto& u : adj[v]) if(u != p) dfs(u, v,
  adj);
26        tout[v] = ++timer;
27    }
28
29    bool isAncestor(int u, int v){ return tin[u] <=
  tin[v] & tout[u] >= tout[v]; }
30
31    int lca(int u, int v){
32        if(isAncestor(u, v)) return u;
33        if(isAncestor(v, u)) return v;
34
35        for(int i = l ; i >= 0 ; i--){
36            if(!isAncestor(up[u][i], v)) u = up[u][i
  ];

```

```

37     }
38     return up[u][0];
39 }
40 };

```

7.2 Knapsack

```

1 // Knapsack problem implementation using dynamic
  programming
2 // Time complexity: O(n*w), where n is the number of
  items and w is the maximum weight
3 // Space complexity: O(n*w) for the dp table
4
5 int knapsack(int w, vector<ii>& items){
6     int n = items.size();
7     vector<vi> dp(n+1, vi(w+1, 0));
8
9     for(int i = 1 ; i <= n ; i++){
10         for(int j = 0 ; j <= w ; j++){
11             if(items[i-1].first > j) dp[i][j] = dp[i-1][j];
12             else dp[i][j] = max(dp[i-1][j], dp[i-1][j-
              items[i-1].first] + items[i-1].second);
13         }
14     }
15
16     return dp[n][w];
17 }

```

7.3 Coins Change

```

1 // Find the minimum number of coins needed to make a
  given amount using a set of coin denominations
2 // Time complexity: O(n * m) where n is the amount
  and m is the number of coin denominations
3
4 int coinChange(const vector<int>& coins, int amount)
  {
5     vector<int> dp(amount + 1, INT_MAX);
6     dp[0] = 0;
7
8     for (int i = 1; i <= amount; i++) {
9         for (const int& coin : coins) {
10             if (coin <= i && dp[i - coin] != INT_MAX)
11                 {
12                     dp[i] = min(dp[i], dp[i - coin] + 1);
13                 }
14             }
15         }
16     }
17 }

```

```

12     }
13 }
14 }
15
16     return dp[amount] == INT_MAX ? -1 : dp[amount];
17 }

```

7.4 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity: O(n log n) where n is the length
  of the input array
3
4 int lis(vector<int>& arr) {
5     vector<int> dp;
6     for (int num : arr) {
7         auto it = lower_bound(dp.begin(), dp.end(),
              num);
8         if (it == dp.end()) {
9             dp.push_back(num);
10        } else {
11            *it = num;
12        }
13    }
14    return dp.size();
15 }

```

7.5 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm
2 // Time complexity: O(n) where n is the length of the
  input array
3
4 int maxSubArraySum(const vector<int>& arr) {
5     int maxSum = arr[0];
6     int maxLocal = arr[0];
7
8     for (int i = 0; i < arr.size(); i++) {
9         maxLocal = max(arr[i], maxLocal + arr[i]);
10        maxSum = max(maxSum, maxLocal);
11    }
12
13    return maxSum;
14 }

```